

Exemplo Completo - Modulo 2: POO com Decoradores

Classe base: Personagem (abstrata, com encapsulamento, metodos de classe e estaticos)

```
from abc import ABC, abstractmethod

class Personagem(ABC):
    def __init__(self, nome, vida):
        self.nome = nome
        self.__vida = vida

    def mostrar_status(self):
        return f"{self.nome} tem {self.__vida} de vida."

    def receber_dano(self, dano):
        self.__vida -= dano
        print(f"{self.nome} recebeu {dano} de dano!")

    @staticmethod
    def calcular_bonus(forca_base, multiplicador):
        return forca_base * multiplicador

    @classmethod
    def criar_por_texto(cls, texto):
        nome, vida = texto.split(",")
        return cls(nome.strip(), int(vida))

    @abstractmethod
    def atacar(self):
        pass
```

Subclasses Guerreiro e Mago implementando o metodo atacar

```
class Guerreiro(Personagem):
    def atacar(self):
        dano = Personagem.calcular_bonus(10, 1.2)
        print(f"{self.nome} ataca com espada! Causa {dano} de dano.")
        return dano

class Mago(Personagem):
```

Exemplo Completo - Modulo 2: POO com Decoradores

```
def atacar(self):
    dano = Personagem.calcular_bonus(8, 1.5)
    print(f"{self.nome} lança magia! Causa {dano} de dano.")
    return dano
```

Execucao do combate: criando personagens, atacando e mostrando status

```
guerreiro = Guerreiro.criar_por_texto("Thor,100")
mago = Mago.criar_por_texto("Merlin,80")

print(guerreiro.mostrar_status())
print(mago.mostrar_status())

dano = mago.atacar()
guerreiro.receber_dano(dano)

dano = guerreiro.atacar()
mago.receber_dano(dano)

print(guerreiro.mostrar_status())
print(mago.mostrar_status())
```

Resumo dos conceitos aplicados:

- Classe e instância
- Encapsulamento com __vida
- Herança com Guerreiro e Mago
- Polimorfismo com metodo atacar
- Metodo estatico para calcular dano
- Metodo de classe para criar personagem por texto
- Classe abstrata com metodo obrigatorio